Introduction to Transformers

What is a Transformer?

A **Transformer** is a deep learning architecture designed to understand relationships between words in a sequence using something called **Attention Mechanism**.

It became famous after the 2017 paper:

> "Attention Is All You Need"

published by Google.

Simple Definition

A Transformer is an AI model that:

- reads input data,
- understands context,
- and generates meaningful output.

Example

Input:

> "I love artificial intelligence"

Transformer understands:

- "love" relates to "artificial intelligence"
- "I" is the subject
- overall meaning = positive statement about AI

Then it can:

- translate it
- summarize it
- answer questions about it
- generate similar text

Real-Life Analogy

Imagine reading a sentence:

> "The animal didn't cross the road because IT was too tired."

You instantly know:

- "it" = animal

How?

Because your brain pays attention to relationships between words.

Transformers do the same thing mathematically using **Attention**.

Why Were Transformers Created?

Before Transformers, people used:

- RNNs
- LSTMs

for sequence tasks like:

- translation
- speech recognition
- chatbots

But RNNs had major problems.

Problems with RNNs

1. Sequential Processing

RNN reads words one by one:

I → love → AI

This is slow because processing cannot happen in parallel.

---

2. Long-Term Memory Problem

Example:

> "The boy who lived in France for 10 years finally returned because HE missed his family."

RNN may forget who "he" refers to because the important word is far away.

This is called: **Vanishing Gradient Problem**

Transformers Solved This

Transformers introduced:

Self-Attention

which allows the model to:

- look at ALL words together
- understand relationships instantly
- process everything in parallel

Core Idea of Transformers

The core idea is:

> Every word should be able to look at every other word.

Example:

Sentence:

"The cat sat on the mat"

When processing "sat", the model can directly look at:

- cat
- mat
- on

and understand context.

Why Transformers Became Revolutionary

Transformers improved:

Feature	RNN	Transformer
Speed	Slow	Fast
Parallel Processing	No	Yes
Long-range understanding	Weak	Strong
GPU usage	Poor	Excellent
Scalability	Limited	Massive

Applications of Transformers

Transformers power:

Application	Example
Chatbots	ChatGPT
Translation	Google Translate
Search	Google Search
Coding AI	GitHub Copilot
Image AI	Vision Transformers
Speech AI	Voice assistants

Famous Transformer Models

Model	Company
BERT	Google
GPT	OpenAI
Claude	Anthropic
Gemini	Google
Llama	Meta

Important Technical Definition

A Transformer is:

> A neural network architecture based primarily on self-attention mechanisms instead of recurrence.

Very Important Interview Question

Why is it called a Transformer?

Because it transforms:

Input Sequence → Output Sequence

Example:

English → French

"How are you?" → "Comment ça va ?"

Key Components of a Transformer

The Transformer contains:

1. Input Embedding
2. Positional Encoding
3. Encoder
4. Decoder
5. Self-Attention
6. Multi-Head Attention
7. Feed Forward Networks
8. Softmax Layer

We'll learn each separately in detail.

Architecture Overview

Transformer has 2 main blocks:

INPUT → ENCODER → DECODER → OUTPUT

The encoder understands input.

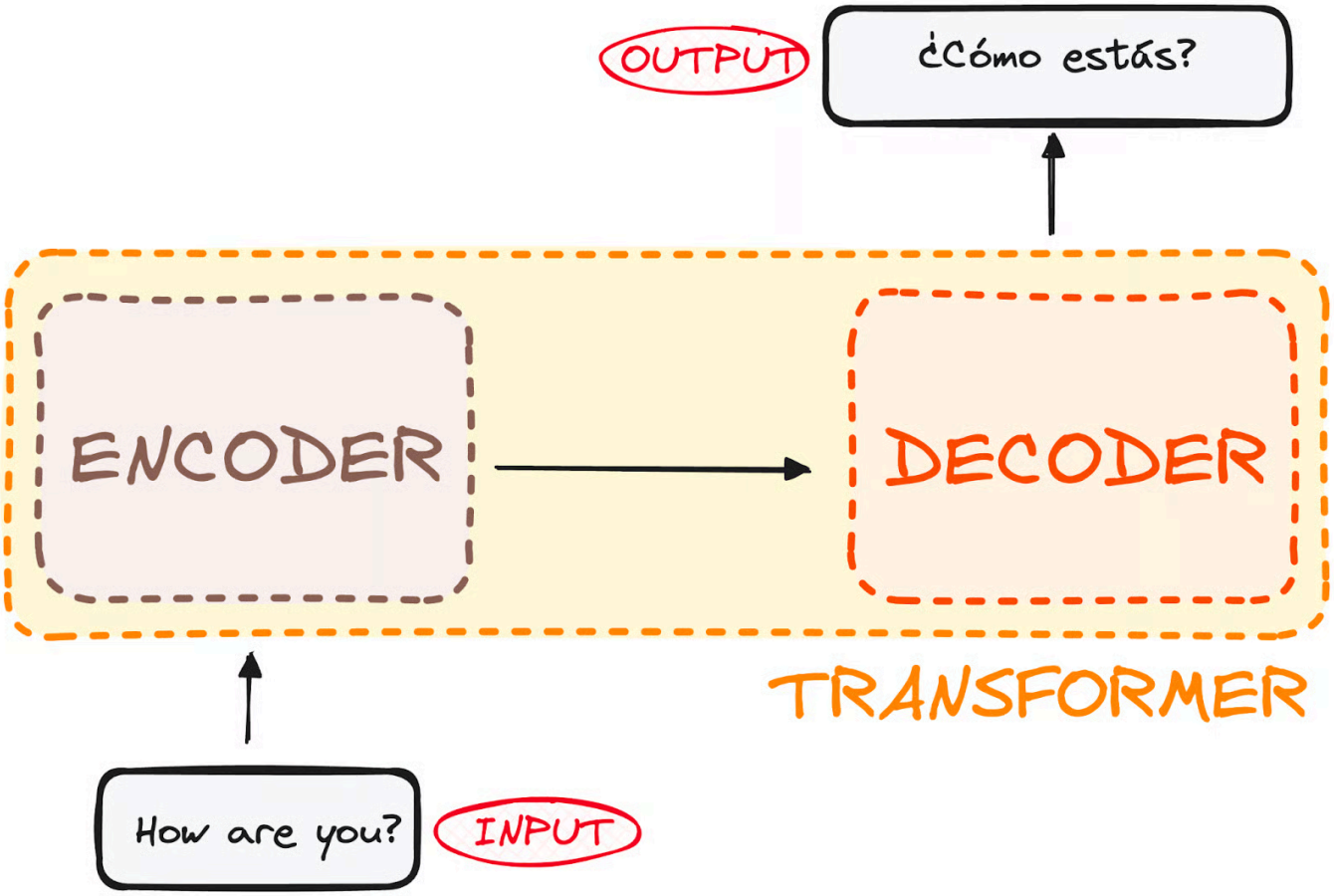
The decoder generates output.

Encoder = Reader

Reads and understands the sentence.

Decoder = Writer

Writes the final answer.



Mini Example

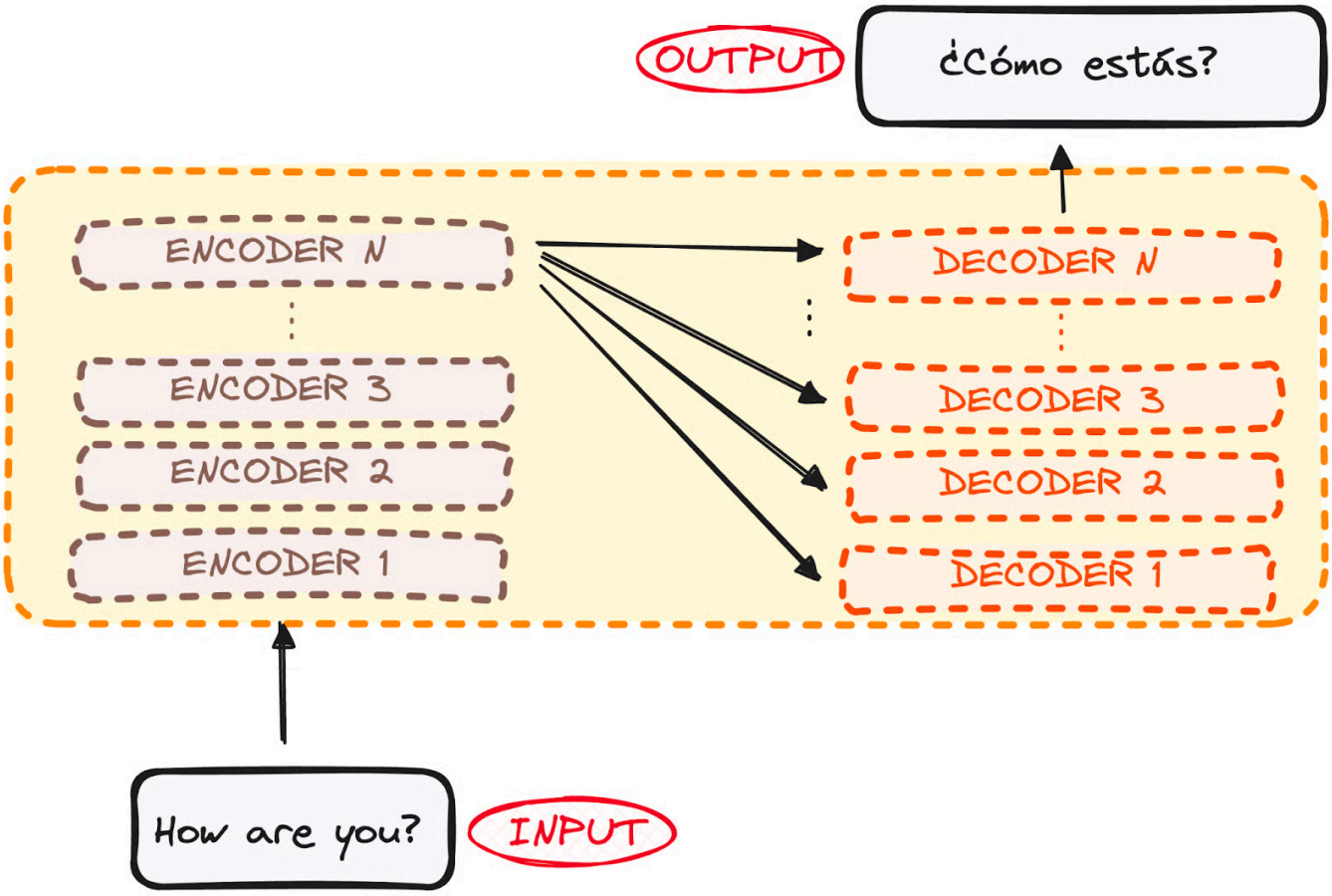
Input:

"How are you?"

Encoder understands meaning.

Decoder generates:

"¿Cómo estás?"



THE ENCODER WORKFLOW (Heart of the Transformer)

This is one of the MOST IMPORTANT sections in Transformers.

If you fully understand the encoder, you understand almost 70% of Transformer architecture.

What is an Encoder?

The encoder is the understanding part of the Transformer.

Its job is:

Input Sentence → Meaningful Context Representation

Simple Definition

The encoder reads the sentence and creates a contextual understanding of every word.

Example

Sentence:

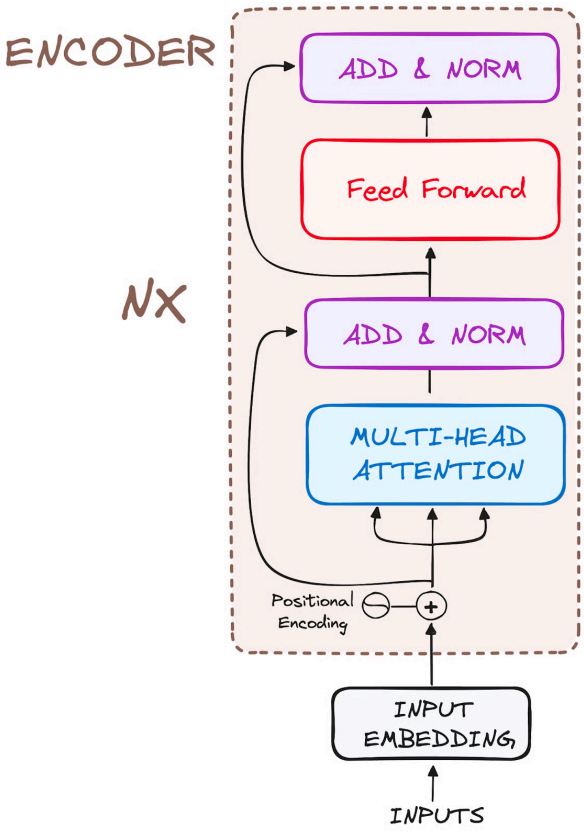
"The bank is near the river"

Encoder understands:

- "bank" means river bank
  NOT:
- bank account

because it analyzes nearby words like: **river**

This is called: Contextual Understanding



Real-Life Analogy

Imagine reading:

> "Apple released a new iPhone."

You instantly know:

- Apple = company

NOT fruit.

Why?

Because your brain uses context.

The encoder does the same thing mathematically.

Overall Encoder Flow

The encoder workflow is:

Input Tokens

↓

Input Embeddings

↓

Positional Encoding

↓

Multi-Head Self Attention

↓

Add & Normalize

↓

Feed Forward Network

↓

Add & Normalize

↓

Output Context Vectors

Big Picture of Encoder

The encoder contains multiple identical layers.

Original Transformer:

- 6 Encoder Layers

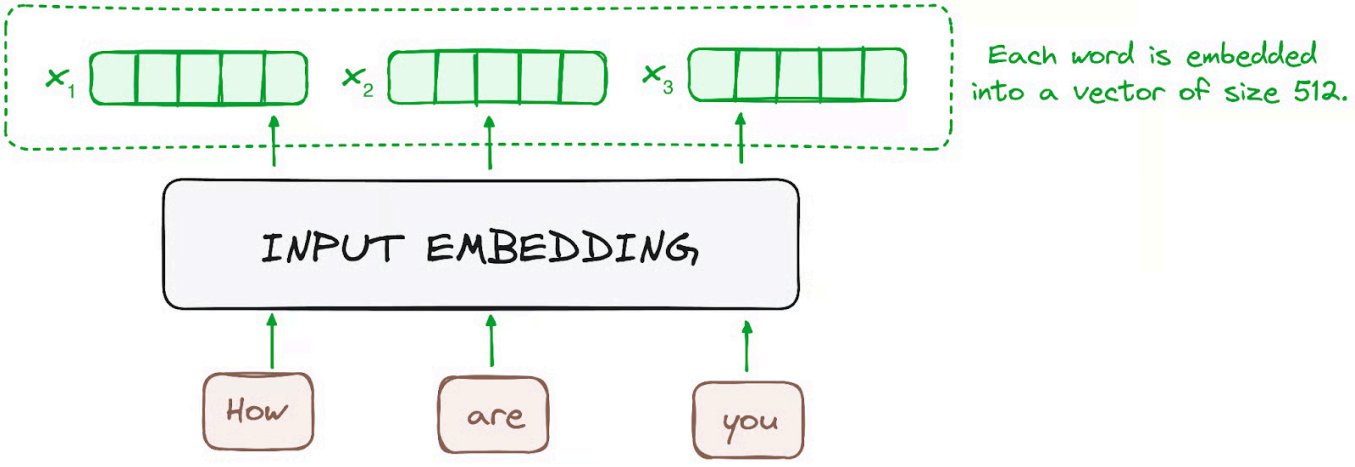
Each layer improves understanding more deeply.

STEP 1 – INPUT EMBEDDINGS

What is Embedding?

Computers cannot understand words directly.

So words are converted into numbers called: **Vectors / Embeddings**



Example

Sentence:

"I love AI"

After tokenization:

["I", "love", "AI"]

Embedding layer converts them into vectors:

"I" → [0.2, 0.8, 0.1, ...]

"love" → [0.9, 0.1, 0.7, ...]

"AI" → [0.5, 0.4, 0.6, ...]

Why Embeddings Matter

Embeddings capture semantic meaning.

Example:

King ≈ Queen

Dog ≈ Puppy

Similar words have similar vectors.

Original Transformer used:

512-dimensional vectors

That means each word becomes:

Vector size = 512

Key Note

Only the FIRST encoder receives actual word embeddings.

Other encoder layers receive outputs from previous encoder layers.

Simple Analogy

Embedding is like:

Word → Mathematical Identity Card

STEP 2 — POSITIONAL ENCODING

Problem Without Positional Encoding

Transformers process all words simultaneously.

So they DON'T naturally know:

- first word
- second word
- sentence order

Example

These two sentences are different:

"Dog bites man"

"Man bites dog"

Without positional information:

Transformer may think both are similar.

Solution = Positional Encoding

We add position information to embeddings.

Formula Used

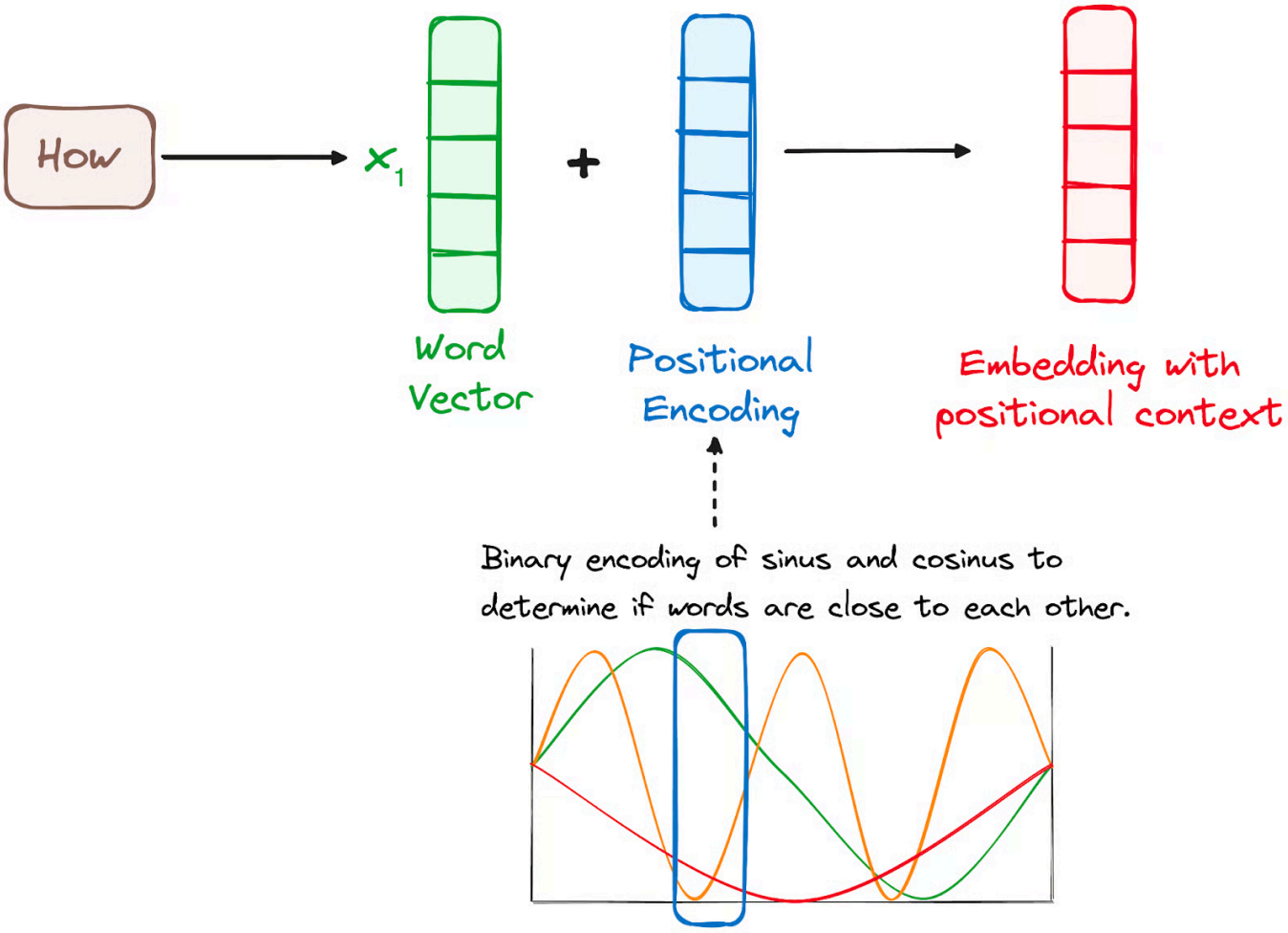
Transformers use sine and cosine functions.

The positional encoding creates unique patterns for every position.

Important Idea

Each word gets:

Final Input = Word Embedding + Positional Encoding



Example

Suppose:

Embedding("cat") = [0.2, 0.5]

Position Encoding(2) = [0.1, 0.9]

Final vector:

[0.2,0.5] + [0.1,0.9]

=

[0.3,1.4]

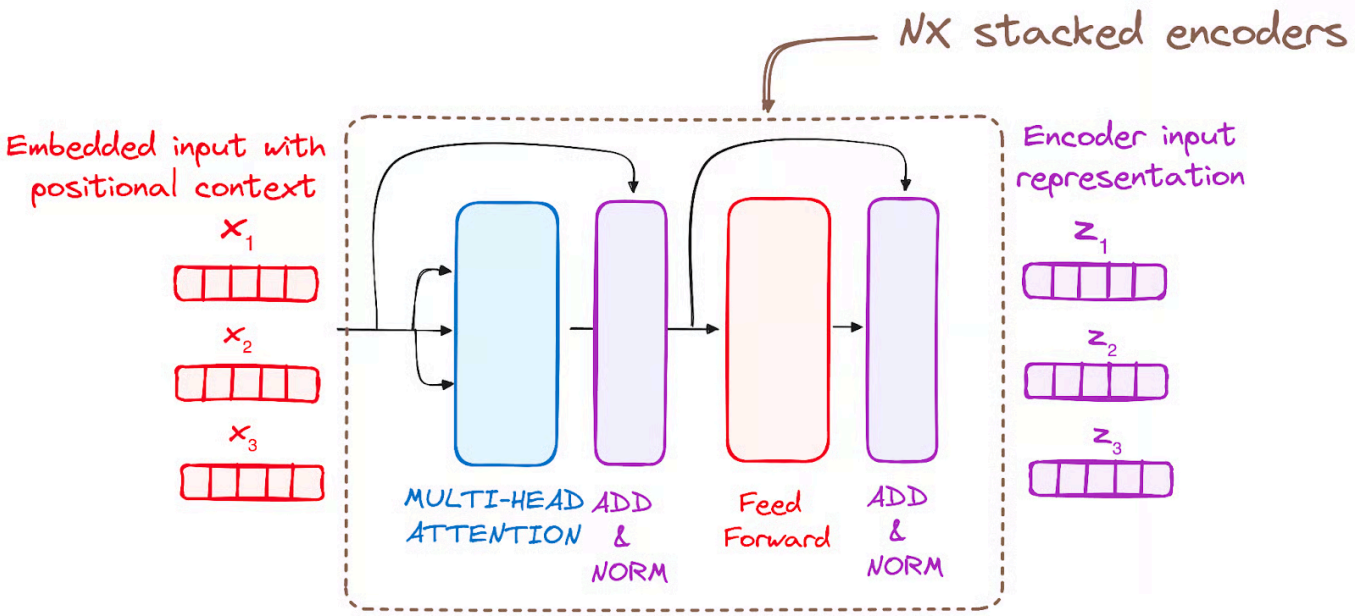
Why Sine & Cosine?

Because they:

- create unique position patterns
- generalize to long sequences
- preserve relative distances

STEP 3 — STACK OF ENCODER LAYERS

Now the real intelligence begins.



Structure of One Encoder Layer

Each encoder layer contains:

1. Multi-Head Self Attention
2. Add & Normalize
3. Feed Forward Network
4. Add & Normalize

Visual Flow

Input

↓

Multi-Head Attention

↓

Add & Normalize

↓

Feed Forward Network

↓

Add & Normalize

↓

Output

STEP 3.1 – SELF-ATTENTION

This is the MOST IMPORTANT concept in Transformers.

Self-Attention = Core of Transformers

Main Idea

Every word looks at every other word.

Example

Sentence:

"The animal didn't cross the road because it was tired"

When processing:

"it"

the model learns:

"it" → refers to "animal"

Attention Mechanism Components

Self-attention uses:

Component	Meaning
Query (Q)	What am I looking for?
Key (K)	What information do I contain?
Value (V)	Actual information

Real-Life Analogy

Think of YouTube search.

Query

Your search text.

Key

Video titles.

Value

Actual video content.

The best matching title gets highest attention.

How Self-Attention Works

Step 1 — Create Q, K, V

Each word creates:

- Query vector
- Key vector
- Value vector

using learned matrices.

Example

Word:

"apple"

becomes:

Q = [1,2,3]

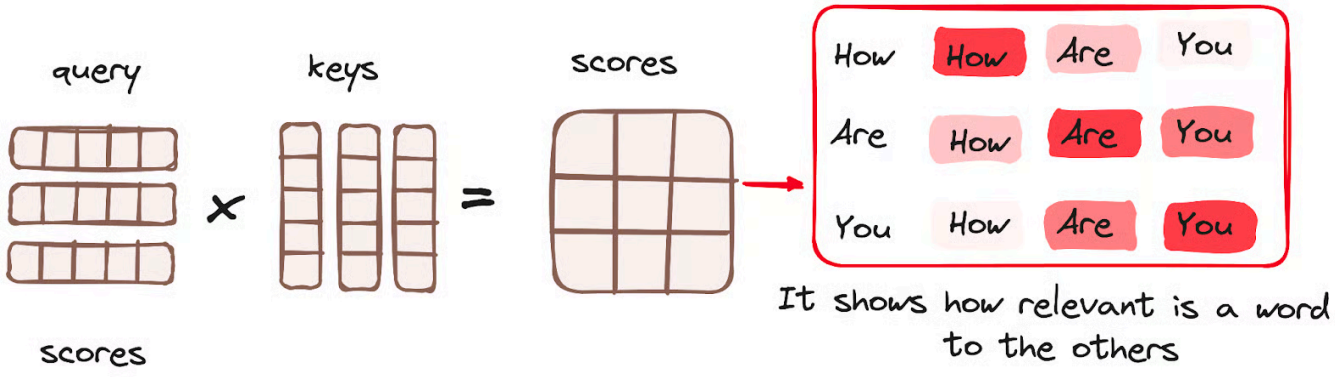
K = [0,1,1]

V = [5,6,7]

Step 2 – Calculate Attention Scores

We compare:

Query × Key



using dot product.

This tells:

How important is another word?

Mathematical Formula

The core attention equation is:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d})V$$

Meaning of Formula

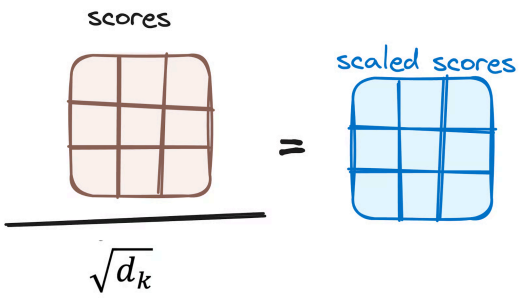
Part	Meaning
QK^T	Similarity score
$\sqrt{d_k}$	Scaling factor
Softmax	Converts to probabilities
V	Final.weighted information

Step 3 – Scaling

Scores become very large sometimes.

So we divide by:

$\sqrt{d_k}$



This stabilizes gradients.

Step 4 – Softmax

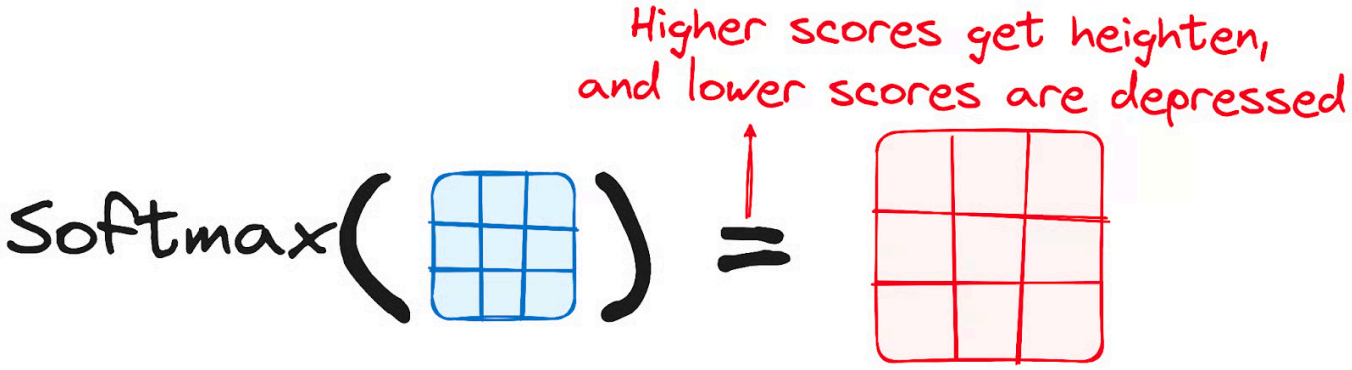
Softmax converts scores into probabilities:

[2.1, 1.3, 0.2]

↓

[0.7, 0.2, 0.1]

Higher score = more attention.



Step 5 – Multiply with Values

Attention weights multiply with value vectors.

Important words get amplified.

Unimportant words get reduced.

Final Output

The output becomes:

Context-aware representation

MULTI-HEAD ATTENTION

Why Multiple Heads?

One attention head may learn:

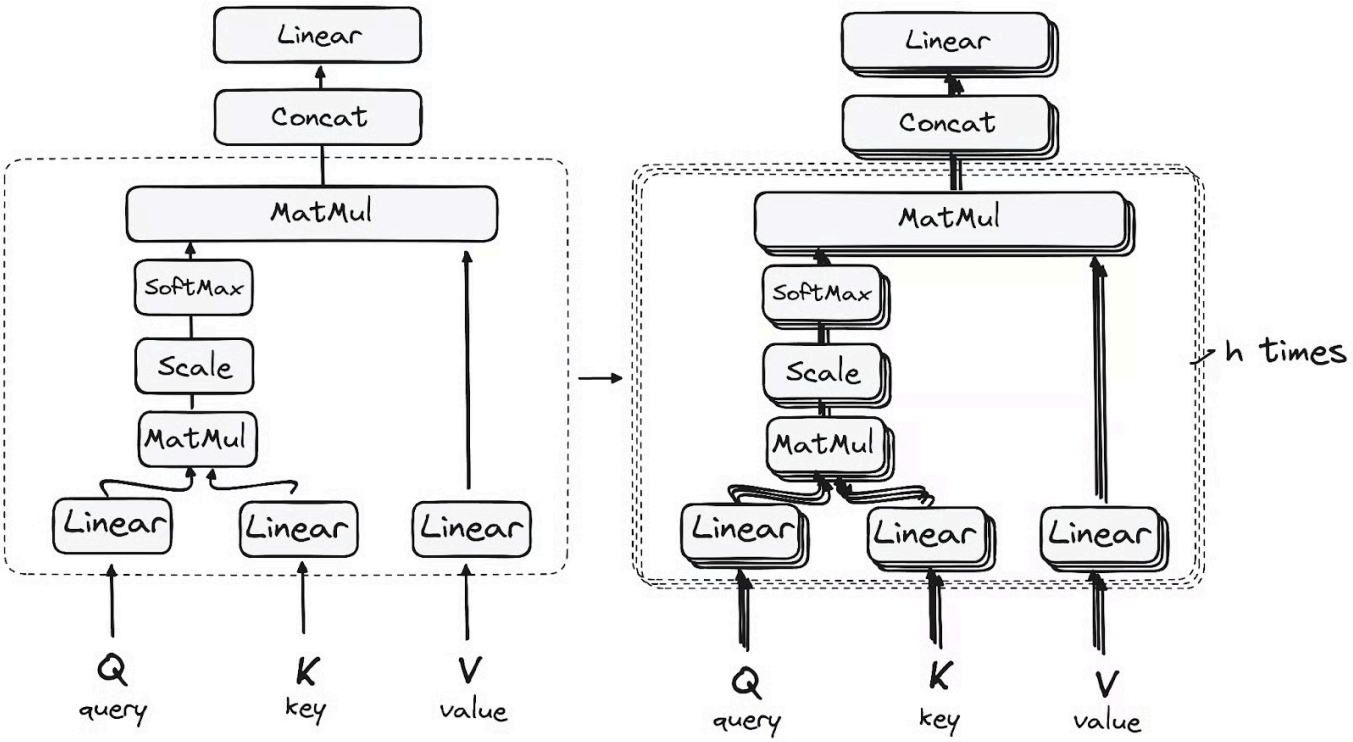
- grammar

Another may learn:

- subject-object relation

Another may learn:

- semantic meaning



Example

Sentence:

"The boy kicked the ball"

Different heads may focus on:

Head	Learns
Head 1	boy ↔ kicked

Head 2 kicked ↔ ball

Head 3 Sentencestructure

Important Point

Attention happens multiple times in parallel.

Then outputs are combined.

This is: Multi-Head Attention

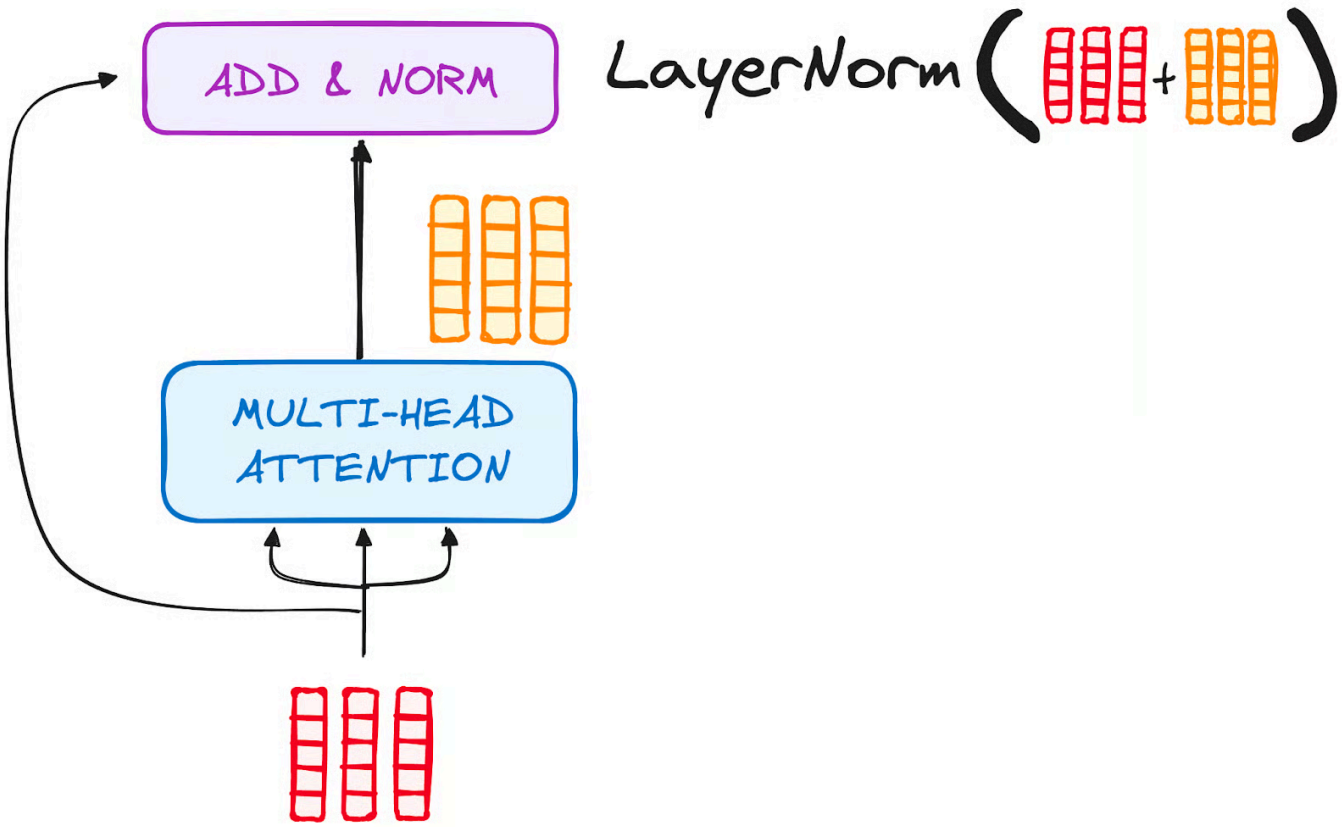
STEP 3.2 — RESIDUAL CONNECTION + NORMALIZATION

Residual Connection

We add input back to output.

Formula:

Output = Layer(x) + x



Why?

Helps prevent:

Vanishing Gradient Problem

Allows deep networks.

Layer Normalization

Normalizes values for stable learning.

Benefits:

- faster training
- stable gradients
- better convergence

Simple Analogy

Residual connection is like:

"Keep original notes while improving them"

STEP 3.3 — FEED FORWARD NETWORK (FFN)

What is FFN?

A small neural network applied independently to every token.

Structure

Usually:

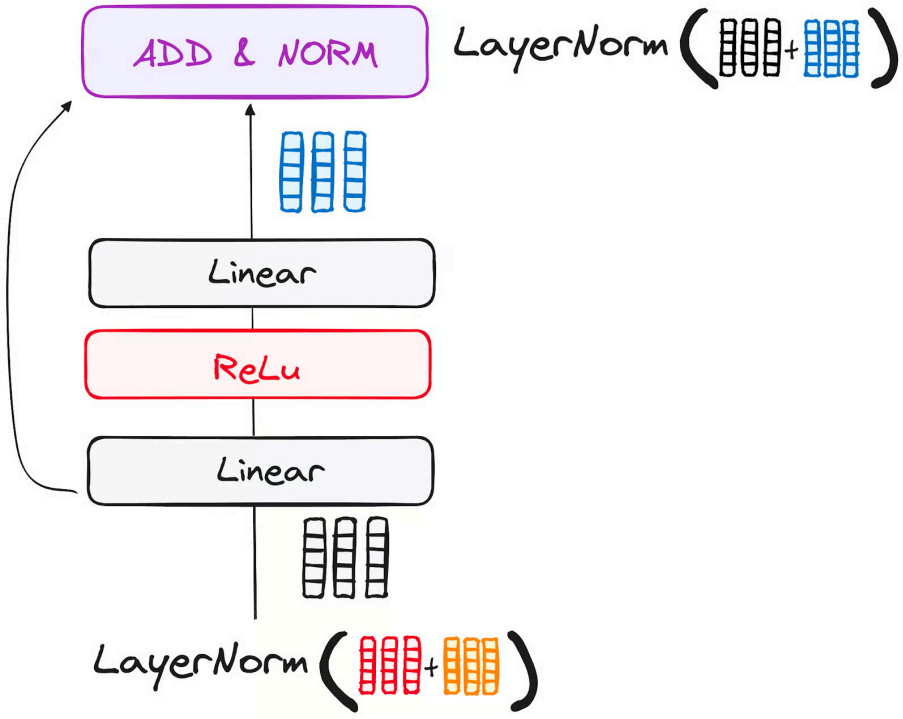
Linear

↓

ReLU

↓

Linear



Formula

FFN(x)= max(0,xW1+b1)W2+b2

Purpose

FFN helps:

- learn deeper features
- refine representations
- improve abstraction

Simple Analogy

Attention = gathering information

FFN = processing information deeply

STEP 4 — OUTPUT OF ENCODER

After all encoder layers:

we get:

Contextualized Representations

Every word now understands:

- grammar
- meaning
- relationships
- sentence context

Example

Input:

"The bank near the river"

Encoder output understands:

bank = river bank

NOT financial bank.

PART 3 – DECODER WORKFLOW (Text Generation Engine)

Now we move to the second major component of Transformers:

The Decoder

If the encoder is the reader, then the decoder is the writer.

The decoder generates the final output sequence word by word.

What is the Decoder?

The decoder is responsible for:

Generating output tokens one by one

Example

Input:

"How are you?"

Encoder understands meaning.

Decoder generates:

"¿Cómo estás?"

step by step.

Main Goal of Decoder

The decoder:

- reads encoder output
- looks at previously generated words
- predicts next word

Real-Life Analogy

Imagine translating English to Spanish.

Encoder

Understands the English sentence.

Decoder

Writes the Spanish sentence word by word.

Important Concept

The decoder works:

Autoregressively

Meaning:

Previous words → predict next word

Example of Autoregressive Generation

Suppose output sentence is:

"I love AI"

Generation happens like:

START → "I"

"I" → "love"

"I love" → "AI"

"I love AI" → END

Overall Decoder Workflow

Previous Output Tokens

↓

Output Embedding

↓

Positional Encoding

↓

Masked Multi-Head Attention

↓

Add & Normalize

↓

Cross Attention

↓

Add & Normalize

↓

Feed Forward Network

↓

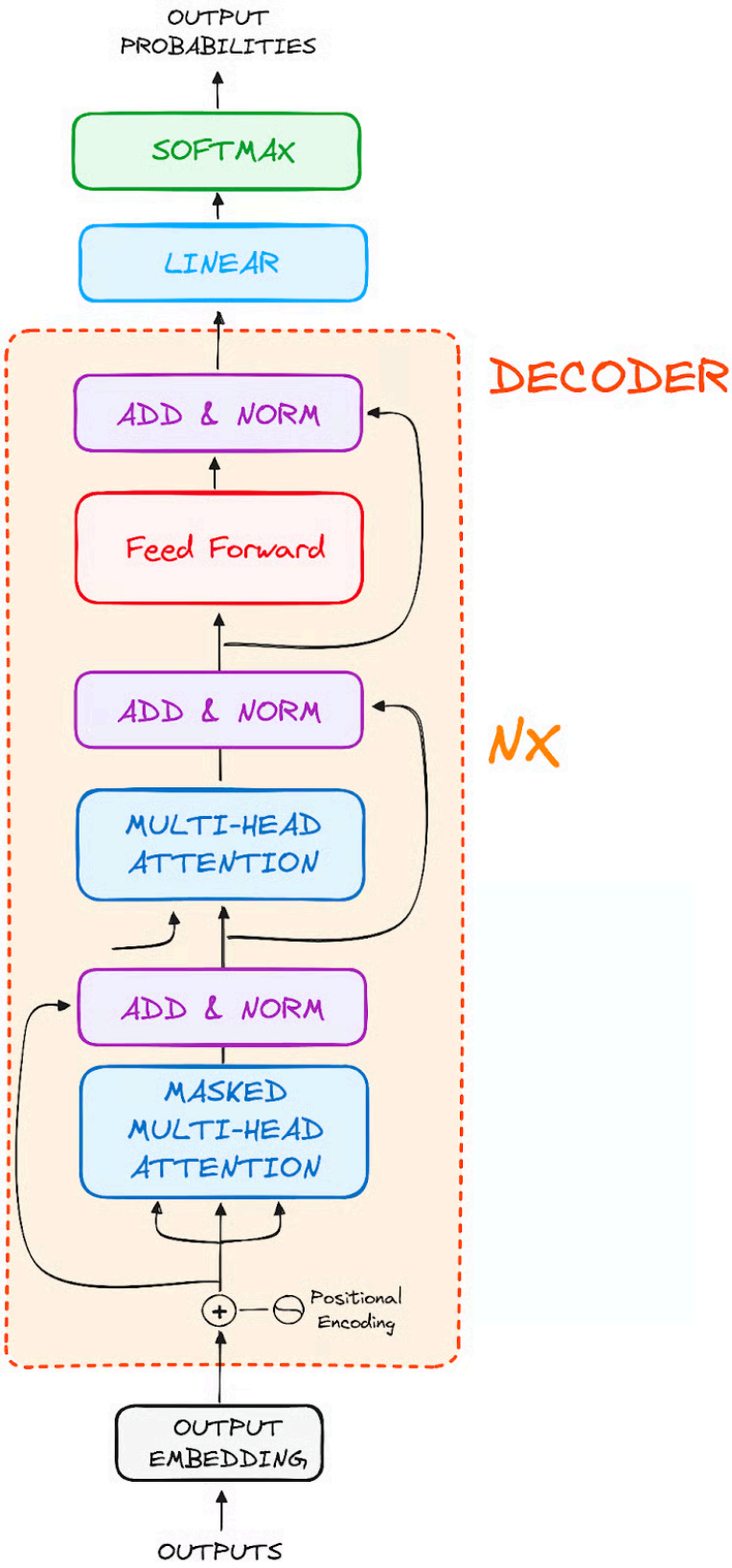
Add & Normalize

↓

Linear + Softmax

↓

Next Word Prediction


OUTPUT PROBABILITIES
SOFTMAX
LINEAR
DECODER
ADD & NORM
Feed Forward
ADD & NORM
NX
MULTI-HEAD ATTENTION
ADD & NORM
MASKED MULTI-HEAD ATTENTION
Positional Encoding
OUTPUT EMBEDDING
OUTPUTS

Decoder Main Components

Component	Purpose
Output Embedding	Convert output words into vectors
Positional Encoding	Add sequence order
Masked Self-Attention	Prevent cheating from future words
Cross Attention	Connect encoder + decoder
FeedForward Network	Deep processing
Linear Layer	Vocabulary scoring
Softmax	Probability generation

STEP 1 – OUTPUT EMBEDDINGS

What Happens Here?

Just like encoder input embeddings:

decoder converts output tokens into vectors.

Example

Suppose decoder already generated:

"I love"

Embeddings:

"I" → [0.3,0.1,...]

"love"→ [0.8,0.5,...]

Important Point

Decoder uses:

- previously generated outputs
- as new inputs

STEP 2 – POSITIONAL ENCODING

Why Needed?

Decoder also processes tokens in parallel.

So it must know word order.

Formula

Same as encoder:

Final Vector = Embedding + Positional Encoding

Example

Sentence:

"I love AI"

Position info:

Word	Position
I	1
love	2
AI	3

Important Point

Encoder and decoder BOTH use positional encoding.

STEP 3 – STACK OF DECODER LAYERS

Decoder contains multiple identical layers.

Original Transformer:

6 Decoder Layers

Structure of One Decoder Layer

Each decoder layer has:

1. Masked Self-Attention
2. Cross Attention
3. Feed Forward Network

Visual Flow

Input

↓

Masked Self Attention

↓

Add & Normalize

↓

Cross Attention

↓

Add & Normalize

↓

Feed Forward Network

↓

Add & Normalize

↓

Output

STEP 3.1 — MASKED SELF-ATTENTION

This is VERY IMPORTANT.

What is Masking?

The decoder must NOT see future words.

Example

Suppose target sentence:

"I love AI"

When predicting:

"love"

decoder should only see:

"I"

NOT:

"AI"

Otherwise model cheats.

Why is This Important?

During training:

entire sentence is available.

Without masking:

model could simply copy future words.

That would destroy learning.

Masking Concept

Future positions are hidden using a mask matrix.

Example Mask Matrix

For sentence length = 4

[1 0 0 0

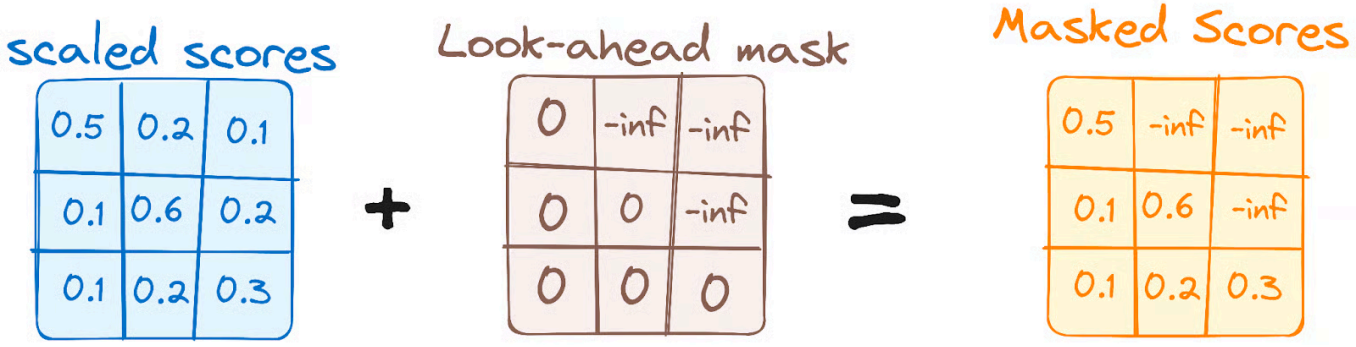
1 1 0 0

1 1 1 0

1 1 1 1]

Meaning

Row	Visible Words
Word 1	Only itself
Word 2	Words 1–2
Word 3	Words 1–3
Word 4	Words 1–4



Important Concept

Decoder prediction depends ONLY on:

Past words

Never future words.

Self-Attention Inside Decoder

Works similarly to encoder attention:

- Query
- Key
- Value

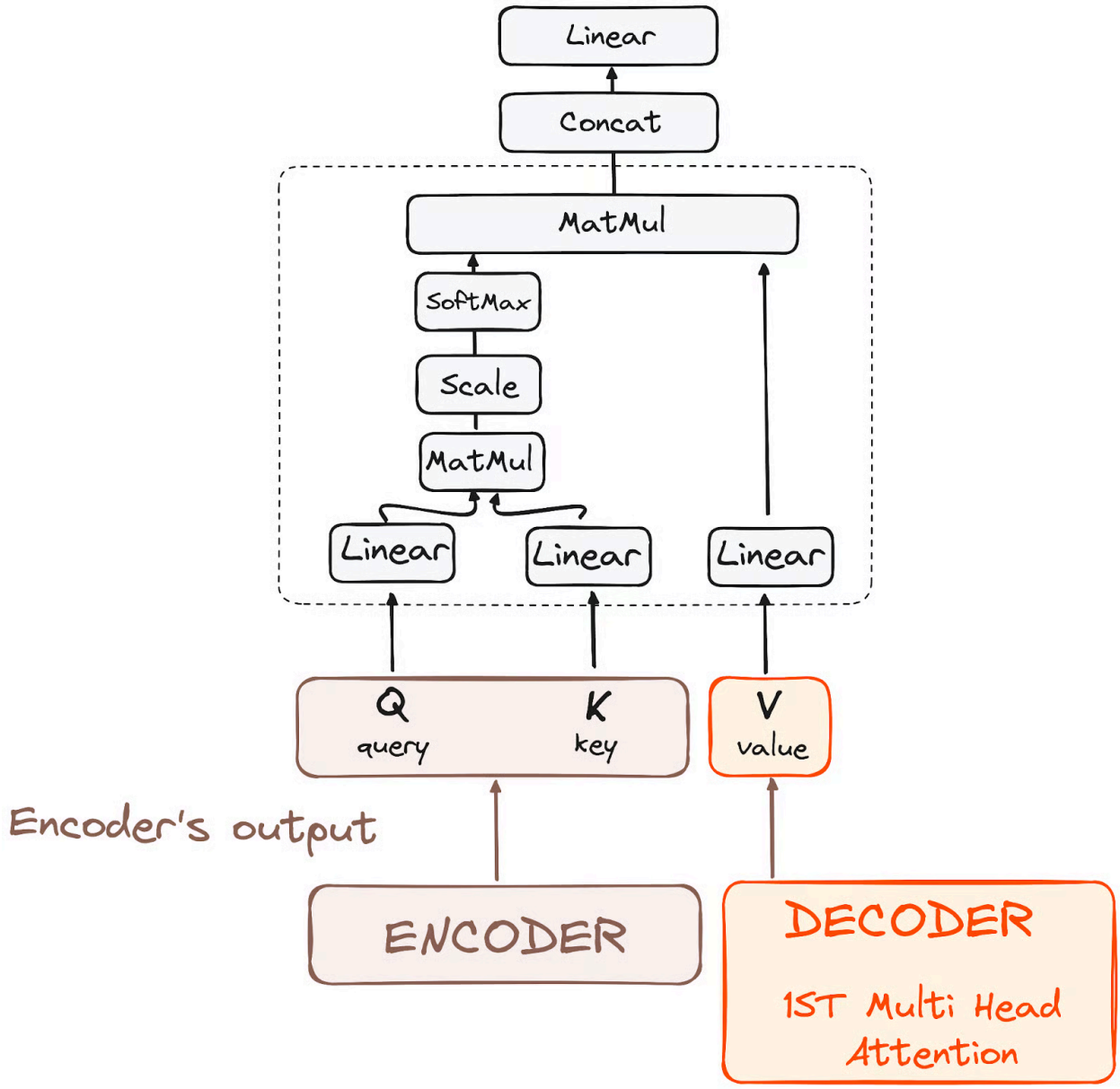
But with masking applied.

STEP 3.2 — CROSS ATTENTION (Encoder-Decoder Attention)

This is where:

Encoder and Decoder communicate.

Main Idea



Decoder asks encoder:

"Which input words are important right now?"

Example Translation

Input:

"How are you?"

Output generation:

"¿Cómo estás?"

When generating:

"estás"

decoder focuses on:

"are you"

from encoder output.

Important Technical Detail

In cross attention:

Component	Comes From

Query	Decoder
Key	Encoder
Value	Encoder

Core Idea

Decoder uses encoder information to generate relevant output.

Real-Life Analogy

Think of:

Encoder = textbook

Decoder = student writing answers

Student looks back at textbook while writing.

That is cross attention.

STEP 3.3 — FEED FORWARD NETWORK

Same as encoder FFN.

Structure:

Linear

↓

ReLU

↓

Linear

Purpose

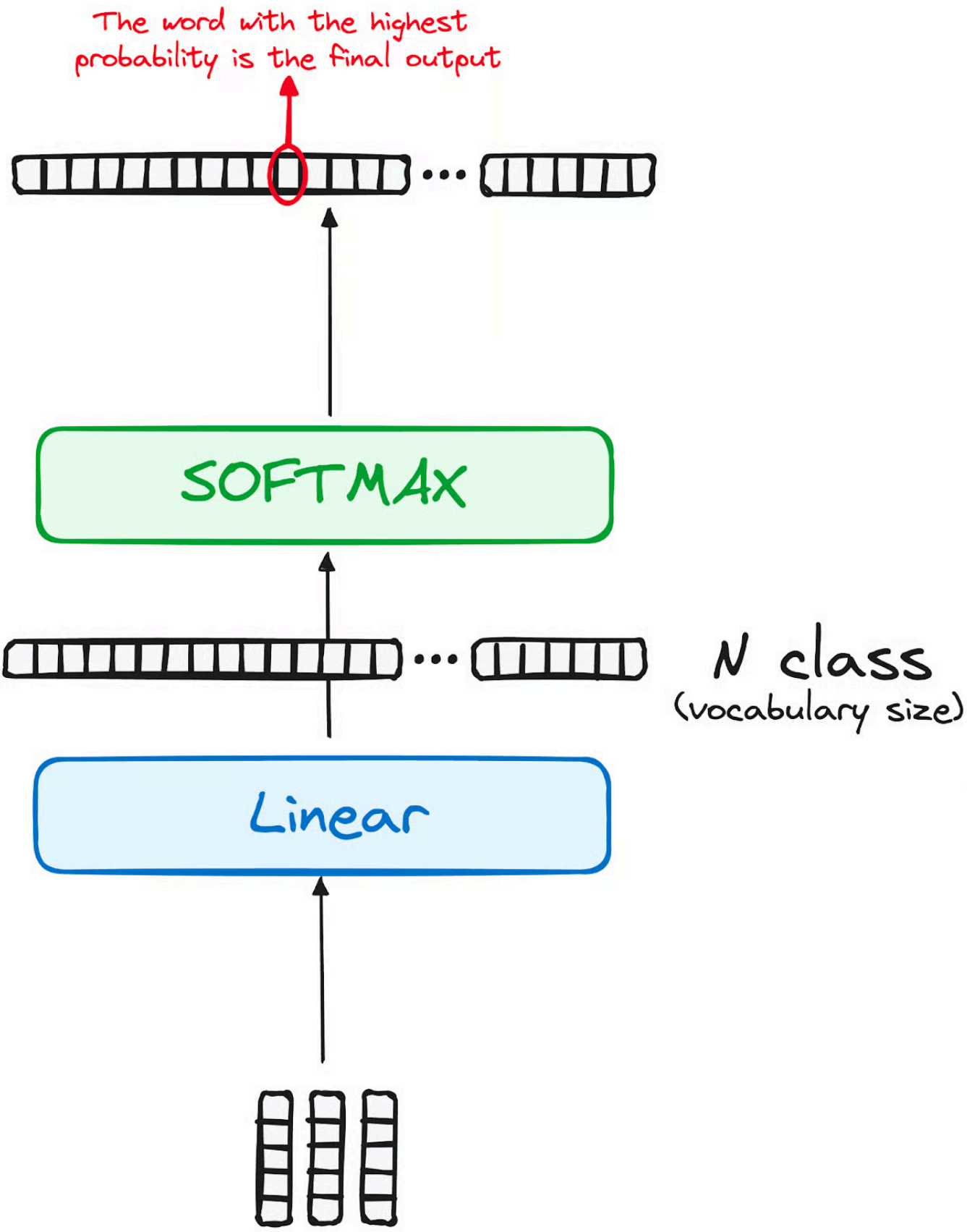
Further processing and refinement.

Formula

$$FFN(x)=max(0,xW1+b1)W2+b2$$

STEP 4 – LINEAR LAYER + SOFTMAX

Now decoder predicts next word.


The word with the highest probability is the final output
SOFTMAX
N class
(vocabulary size)
Linear

Linear Layer

Transforms decoder output into vocabulary scores.

Suppose vocabulary contains:

10000 words

Then output becomes:

10000 scores

Example

Suppose scores are:

cat → 2.5

dog → 1.2

apple → 0.3

Softmax Layer

Softmax converts scores into probabilities.

Formula

$$\text{Attention}(Q, K, V) = \text{Softmax}(QK^T / \sqrt{d_k})V$$

Example

After softmax:

cat → 0.75

dog → 0.20

apple → 0.05

Final Prediction

Highest probability wins.

Predicted word:

"cat"

END TOKEN

Generation continues until model predicts:

<END>

which means:

Sentence complete

Full Decoder Example

Suppose translation task:

Input:

"I love AI"

Generation Process

START

↓

"I"

↓

"I love"

↓

"I love AI"

↓

END

One word generated at a time.

Decoder vs Encoder

Feature	Encoder	Decoder
Main Role	Understand input	Generate output
Attention Type	Self-attention	Masked + Cross attention
Sees Future Tokens?	Yes	No
Uses Encoder Output?	No	Yes

REAL-LIFE TRANSFORMER MODELS

1. BERT (Google)

Full Form

Bidirectional Encoder Representations from Transformers

Architecture

Encoder Only

Main Idea

Understands text using both left and right context.

Example

"The bank near the river"

BERT understands:

bank = river bank

Applications

- Google Search
- Question Answering
- Text Classification

Memory Trick

BERT = Understands Text

---

2. GPT / ChatGPT (OpenAI)

Full Form

Generative Pretrained Transformer

Architecture

Decoder Only

Main Idea

Generates text one word at a time.

Example

"Once upon a time..."

GPT predicts the next words.

Applications

- ChatGPT
- AI Writing
- Coding Assistants

Memory Trick

GPT = Generates Text

---

3. LaMDA (Google)

Full Form

Language Model for Dialogue Applications

Main Idea

Built for conversations and chat systems.

Applications

- Virtual Assistants
- Conversational AI

Memory Trick

LaMDA = Conversation AI

---

4. Claude (Anthropic)

Main Idea

AI assistant focused on:

- safety
- reasoning
- long context understanding

Special Feature

Constitutional AI

Memory Trick

Claude = Safe AI

---

5. Vision Transformers (ViT)

Main Idea

Processes images using Transformer architecture.

Images are divided into:

small patches

Applications

- Image Classification

- Object Detection

Memory Trick

ViT = Transformer for Images

---

6. Multimodal Transformers

Main Idea

Processes multiple input types together:

- text
- image
- audio
- video

Example Models

- GPT-5
- Gemini
- Llama

Memory Trick

Multimodal = Multiple Input Types

LIMITATIONS OF TRANSFORMERS

- High memory usage
- Expensive training
- Hallucinations
- Long context is computationally expensive

TRANSFORMERS – INTERVIEW QUESTIONS & ANSWERS NOTES

BASIC QUESTIONS

1. What is a Transformer?

A Transformer is a deep learning architecture that uses self-attention mechanisms to process sequential data in parallel.

---

2. Why were Transformers introduced?

Transformers were introduced to solve problems in RNNs such as:

- slow sequential processing
- difficulty handling long-range dependencies

---

3. What is the main innovation in Transformers?

Self-Attention Mechanism

---

4. What is self-attention?

Self-attention allows every word in a sentence to focus on all other words to understand context.

---

5. Why are Transformers faster than RNNs?

Because Transformers process all words in parallel instead of sequentially.

ARCHITECTURE QUESTIONS

6. What are the two main components of a Transformer?

Encoder and Decoder

---

7. What is the role of the Encoder?

Encoder understands the input sentence and creates contextual representations.

---

8. What is the role of the Decoder?

Decoder generates output text word by word.

---

9. What is Input Embedding?

Converting words into numerical vectors.

---

10. Why is Positional Encoding needed?

Because Transformers do not process words sequentially and need position information.

SELF-ATTENTION QUESTIONS

11. What are Query, Key, and Value?

Component	Purpose
Query	What the word is searching for
Key	What information the word contains
Value	Actual information passed forward

---

12. What is Multi-Head Attention?

Using multiple attention mechanisms in parallel to learn different relationships.

---

13. Why is Multi-Head Attention important?

Different heads learn:

- grammar
- semantics
- relationships
- sentence structure

---

14. What is the purpose of Softmax in attention?

Softmax converts attention scores into probabilities.

---

15. Why are attention scores scaled?

To prevent extremely large values and stabilize training.

ENCODER QUESTIONS

16. What are the main parts of an Encoder layer?

- Multi-Head Self Attention
- Feed Forward Network

- Residual Connection
- Layer Normalization

---

17. What is a Feed Forward Network (FFN)?

A small neural network used for deeper processing after attention.

---

18. Why are Residual Connections used?

To prevent vanishing gradient problems and enable deeper networks.

---

19. What is Layer Normalization?

A technique used to stabilize and speed up training.

DECODER QUESTIONS

20. What is Masked Self-Attention?

A decoder mechanism that prevents the model from seeing future words.

---

21. Why is masking important?

To avoid cheating during text generation.

---

22. What is Cross Attention?

Decoder attending to encoder outputs.

---

23. What does autoregressive generation mean?

Generating text one token at a time using previous outputs.

MODEL QUESTIONS

24. What architecture does BERT use?

Encoder Only

---

25. What architecture does GPT use?

Decoder Only

---

26. What is BERT mainly used for?

- Understanding tasks
- Question Answering
- Text Classification

---

27. What is GPT mainly used for?

- Text Generation
- Chatbots
- Content Writing

---

28. What is LaMDA?

A Transformer model designed for conversations and dialogue systems.

---

29. What is Claude known for?

- Safety
- Reasoning
- Long-context understanding

---

30. What are Vision Transformers (ViT)?

Transformers applied to image processing tasks.

COMPARISON QUESTIONS

31. Difference between RNN and Transformer?

RNN	Transformer
Sequential	Parallel
Slow	Fast
Weak long memory	Strong long memory

---

32. Difference between CNN and Transformer?

CNN	Transformer
Best for images	Best for NLP + multimodal
Limited context	Strong context understanding

ADVANCED QUESTIONS

33. What is Flash Attention?

An optimized attention mechanism that reduces GPU memory usage and speeds up training.

---

34. What is RoPE?

Rotary Position Embedding

Used for better long-sequence understanding.

---

35. What is Mixture of Experts (MoE)?

Only a subset of model parameters activate for each task.

---

36. What are Multimodal Transformers?

Models that process:

- text
- image
- audio
- video

together.

LIMITATION QUESTIONS

37. What are the limitations of Transformers?

- High memory usage
- Expensive training
- Hallucinations
- Quadratic complexity

---

38. What is hallucination in AI?

Generating incorrect information confidently.

---

39. Why are Transformers computationally expensive?

Because self-attention compares every token with every other token.

IMPORTANT ONE-LINERS

40. Why is "Attention Is All You Need" famous?

Because it introduced the Transformer architecture.

---

41. Why are Transformers important in AI?

They revolutionized NLP and power modern AI systems like ChatGPT and Claude.

VERY IMPORTANT INTERVIEW QUESTION

42. Explain Transformer workflow in simple terms.

Input

↓

Embedding

↓

Positional Encoding

↓

Self Attention

↓

Encoder Understanding

↓

Decoder Generation

↓

Output Text

PRACTICAL IMPLEMENTATIONS:

```
# ==========================================
# MINI GPT / TRANSFORMER FROM SCRATCH
# Beginner-Friendly Full Implementation
# ==========================================
```

```
# ==========================
# STEP 1 — IMPORT LIBRARIES
# ==========================

import torch
import torch.nn as nn
import torch.optim as optim
import math

print("Libraries Imported Successfully")



# ==========================================
# STEP 2 — SAMPLE TRAINING TEXT
# ==========================================

text = "i love ai and ai loves humans"

print("\nTraining Text:")
print(text)



# ==========================================
# STEP 3 — TOKENIZATION
# ==========================================

tokens = text.split()
print("\nTokens:")
print(tokens)
```

```
# ============================================
# STEP 4 — BUILD VOCABULARY
# ============================================

vocab = sorted(set(tokens))

word_to_idx = {
    word: idx for idx, word in enumerate(vocab)
}

idx_to_word = {
    idx: word for word, idx in word_to_idx.items()
}

print("\nVocabulary:")
print(word_to_idx)


# ============================================
# STEP 5 — CONVERT TOKENS TO IDS
# ============================================

token_ids = [word_to_idx[word] for word in tokens]

print("\nToken IDs:")
print(token_ids)
```

```
# ============================================

# STEP 6 — CREATE INPUT-TARGET PAIRS

# ============================================


# Example:

# input  = i

# target = love


inputs = token_ids[:-1]

targets = token_ids[1:]


inputs = torch.tensor(inputs)

targets = torch.tensor(targets)


print("\nInputs:")

print(inputs)


print("\nTargets:")

print(targets)




# ============================================

# STEP 7 — MODEL HYPERPARAMETERS

# ============================================

vocab_size = len(vocab)

embedding_dim = 16

num_heads = 2

hidden_dim = 32
```

```
print("\nModel Configuration:")
print("Vocabulary Size:", vocab_size)
print("Embedding Dimension:", embedding_dim)



# ============================================
# STEP 8 — POSITIONAL ENCODING
# ============================================

class PositionalEncoding(nn.Module):

    def __init__(self, d_model, max_len=100):

        super().__init__()

        pe = torch.zeros(max_len, d_model)

        position = torch.arange(
            0,
            max_len
        ).unsqueeze(1)

        div_term = torch.exp(
            torch.arange(0, d_model, 2)
            * (-math.log(10000.0) / d_model)
        )

        pe[:, 0::2] = torch.sin(position * div_term)
```

```
        pe[:, 1::2] = torch.cos(position * div_term)

        self.pe = pe.unsqueeze(0)

    def forward(self, x):

        seq_len = x.size(1)

        return x + self.pe[:, :seq_len]



# ============================================
# STEP 9 — MINI TRANSFORMER MODEL
# ============================================

class MiniGPT(nn.Module):

    def __init__(self):

        super().__init__()

        # Embedding Layer
        self.embedding = nn.Embedding(
            vocab_size,
            embedding_dim
        )

        # Positional Encoding
        self.position = PositionalEncoding(
```

```
    embedding_dim
)

# Multi Head Attention
self.attention = nn.MultiheadAttention(
    embed_dim=embedding_dim,
    num_heads=num_heads,
    batch_first=True
)

# Feed Forward Network
self.ffn = nn.Sequential(

    nn.Linear(
        embedding_dim,
        hidden_dim
    ),

    nn.ReLU(),

    nn.Linear(
        hidden_dim,
        embedding_dim
    )
)

# Layer Normalization
self.norm1 = nn.LayerNorm(
    embedding_dim
```

```
    )

    self.norm2 = nn.LayerNorm(
        embedding_dim
    )

    # Final Output Layer
    self.fc = nn.Linear(
        embedding_dim,
        vocab_size
    )

def forward(self, x):

    # Embedding
    x = self.embedding(x)

    # Add Positional Encoding
    x = self.position(x)

    # Self Attention
    attention_output, _ = self.attention(
        x,
        x,
        x
    )

    # Residual + Normalize
    x = self.norm1(
```

```
            x + attention_output
        )

        # Feed Forward
        ffn_output = self.ffn(x)

        # Residual + Normalize
        x = self.norm2(
            x + ffn_output
        )

        # Final Vocabulary Prediction
        output = self.fc(x)

        return output


# ============================================
# STEP 10 — CREATE MODEL
# ============================================

model = MiniGPT()

print("\nMini GPT Model Created Successfully")


# ============================================
# STEP 11 — LOSS FUNCTION
# ============================================
```

```
loss_function = nn.CrossEntropyLoss()


print("\nLoss Function:")

print(loss_function)



# ============================================

# STEP 12 — OPTIMIZER

# ============================================


optimizer = optim.Adam(

    model.parameters(),

    lr=0.01

)


print("\nOptimizer:")

print(optimizer)



# ============================================

# STEP 13 — PREPARE INPUT SHAPE

# ============================================


# batch_size = 1


inputs = inputs.unsqueeze(0)


print("\nInput Shape:")
```

```
print(inputs.shape)



# ==========================================

# STEP 14 — TRAINING LOOP

# ==========================================

print("\nTraining Started...\n")

epochs = 200

for epoch in range(epochs):

    optimizer.zero_grad()

    # Forward Pass
    output = model(inputs)

    # Reshape Output
    output = output.view(
        -1,
        vocab_size
    )

    # Calculate Loss
    loss = loss_function(
        output,
        targets
    )
```

```
    # Backpropagation
    loss.backward()

    # Update Weights
    optimizer.step()

    # Print Loss
    if (epoch + 1) % 20 == 0:

        print(
            f"Epoch {epoch+1}, Loss: {loss.item():.4f}"
        )



# ==========================================
# STEP 15 — TEXT GENERATION
# ==========================================

print("\nText Generation Started...\n")

model.eval()

# Start Word
current_word = "i"

generated_words = [current_word]

# Generate 5 words
```

```
for _ in range(5):

    # Convert Word → ID
    current_id = torch.tensor(
        [[word_to_idx[current_word]]]
    )

    # Model Prediction
    with torch.no_grad():

        output = model(current_id)

    # Get Highest Probability Word
    predicted_id = torch.argmax(
        output[0, -1]
    ).item()

    # Convert ID → Word
    predicted_word = idx_to_word[predicted_id]

    generated_words.append(
        predicted_word
    )

    current_word = predicted_word

# Final Sentence
generated_sentence = " ".join(
    generated_words
```

```
)

print("Generated Sentence:")
print(generated_sentence)

# ==========================================
# STEP 16 — UNDERSTANDING THE FLOW
# ==========================================

print("\n================================")
print("COMPLETE TRANSFORMER FLOW")
print("================================")

print("""
Text
↓
Tokenization
↓
Vocabulary IDs
↓
Embeddings
↓
Positional Encoding
↓
Self Attention
↓
Feed Forward Network
↓
```

Transformer Block

↓

Prediction

↓

Loss Calculation

↓

Backpropagation

↓

Weight Update

↓

Text Generation

""")

PROJECTs TO PRACTICE :

Project	What It Does	Main Skills Learned
1. Next Word Predictor	Predicts the next word from input text	Tokenization, embeddings, softmax, training
2. Mini Chatbot	Simple conversational AI	Text generation, attention, inference
3. AI Text Summarizer	Converts long text into short summaries	Encoder-decoder, sequence understanding
4.>PDFQuestion Answering Bot	Answers questions from uploaded PDFs	RAG, embeddings, vector search
5. ChatGPT Clone	AI assistant for chatting and answering	Transformers, fine-tuning, prompting

After Completing These :

1.Hugging face - Use real pretrained LLMs like GPT2, Llama

2.Fine-Tuning - Train AI on custom datasets

3.RAG (Retrieval Augmented Generation)

4.Vector Databases

5.LoRA / PEFT

6.AI Agents

7.Deployment

THANK YOU